

Vues relationnelles

Géraldine Del Mondo

Département Informatique et Technologies de l'Information

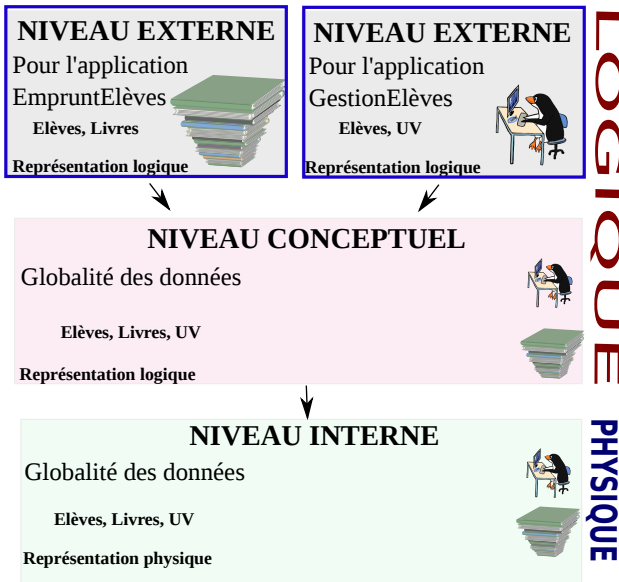
Institut National des Sciences Appliquées - Rouen

geraldine.del_mondo@insa-rouen.fr

Plan...

- 1 Positionnement
- 2 Principe & Définitions
- 3 Expression & Gestion
 - Mise à jour
 - Traitement SGBD
- 4 Droits & Intégrité
- 5 Autre "utilisation"
- 6 Conclusions
 - Vues relationnelles

Indépendance



Offre d'un SGBD

- Niveau physique (→ BD2)
 - Méthodes de placement
 - Index
 - Statistiques
 - Répartition à travers un réseau
- Niveau conceptuel
 - schéma relationnel
 - contrainte d'intégrité
 - procédure stockée / trigger
- Niveau externe
 - **Vues relationnelles**
 - Indépendance données / traitements : une application est indépendante des modifications apportées aux schémas conceptuels et physiques (en théorie).

PRINCIPE & DÉFINITION

Règle des vues

Principe VUES

Chaque (groupe d') utilisateur (ou application) possède sa propre vision de la base

- Certaines informations sont conservées
- Certaines informations du schéma conceptuel disparaissent (projection)
- Certaines informations sont re-structurées (jointure)
- Certaines informations n'existent pas et sont calculées (agrégat : DEGRE_MOYEN)

Schéma **EXEMPLE** : Base COOPERATIVE

VINS (V) (NUM_VIN, CRU, MILLESIME, DEGRE)

VITICULTEURS (VT) (NUM_VITICULTEUR, NOM, PRENOM, VILLE)

PRODUCTIONS (P) (VIN, VITICULTEUR)

BUVEURS (B) (NUM_BUVEUR, NOM, PRENOM, VILLE)

COMMANDES (C) (NUM_COMMANDE, DATE, VIN, QUANTITE, BUVEUR)

EXPEDITION (E) (COMMANDE, DATE, QUANTITE)

EXEMPLES - Schéma

Gestion des vins

VINS (NUM_VIN, CRU, MILLESIME, DEGRE)

DEGRE_MOYEN_CRU (CRU, DEGRE_MOYEN)

Gestion des viticulteurs bordelais

VITIS BORDEAUX (NUM_VITICULTEUR, NOM, PRENOM, VILLE)

Gestion des buveurs parisiens

BUVEURS_PARIS (NUM_BUVEUR, NOM, PRENOM)

COMMANDES_PARIS (NUM_COMMANDE, DATE, BUVEUR, VIN, QUANTITE)

QUANTITE_COMMANDEE_BUVEURS (BUVEUR, QUANTITE)

EXPEDITIONS_PARIS (COMMANDE, DATE, QUANTITE)

Vue



Définition VUE

- Relation dérivée (virtuelle) construite à partir de relations de base et/ou de vues relationnelles
- Une vue est spécifiée par une requête
- Une vue est une fenêtre sur la base de données et elle permet de voir les **mises à jour** entre sessions de travail

EXEMPLE : viticulteurs de vins de Bordeaux

```
CREATE VIEW VITIS_BORDEAUX(NUM_VITICULTEUR,NOM,VILLE)
AS SELECT NUM_VITICULTEUR, NOM, VILLE
FROM VITICULTEURS, VINS, PRODUCTIONS
WHERE NUM_VITICULTEUR = VITICULTEUR AND
      VIN = NUM_VIN AND
      CRU = 'Bordeaux';
```

Cliché

Principe CLICHÉ

- Relation, matérialisée, dont les n-uplets sont définis par le résultat d'une requête sur des relations de base, vues ou clichés
- Un cliché **n'est pas mis à jour** entre sessions de travail - sauf demande expresse -

EXPRESSION & GESTION

Gestion des vues

CRÉATION D'UNE VUE

- **CREATE VIEW** <NOM> **AS** <Requête SQL>

SUPPRESSION D'UNE VUE

- **DROP VIEW** <NOM>

INTERROGATION

- Toujours possible (sous réserve des droits)
- Syntaxe identique à une relation de base

MISE À JOUR

- Non définie dans de nombreux cas
- Syntaxe identique à une mise à jour d'une relation de base (si MAJ possible)

EXEMPLES de vues (1)

Mono relation

```
CREATE VIEW CRUS (NOM)
AS SELECT DISTINCT CRU
FROM VINS
```

Multi-relations

```
CREATE VIEW BUVEURS_BEAUJOLAIS
      (NUM_BUVEUR, NOM, QUANTITE_COMMANDEE)
AS SELECT NUM_BUVEUR, NOM, SUM (QUANTITE)
FROM BUVEURS, VINS, COMMANDES
WHERE NUM_BUVEUR = C.BUVEUR AND
      VIN = NUM_VIN AND
      CRU = 'Beaujolais'
GROUP BY NUM_BUVEUR, NOM
```

Mise à Jour

- Comment reporter la mise à jour de la vue sur les relations de base ?

EXEMPLES

'Monotable' avec **VINS** (NUM_VIN, CRU, MILLESIME, DEGRE)

```
CREATE VIEW VINS_BORDEAUX (NUM_VIN, MILLESIME, DEGRE)
AS SELECT  NUM_VIN, MILLESIME, DEGRE
FROM       VINS
WHERE      CRU = 'Bordeaux' ;
```

Table VINS

NUM_VIN	CRU	MILLESIME	DEGRE
1	Bordeaux	1970	12
2	Bordeaux	1980	14
3	Beaujolais	2010	11

Vue VINS_BORDEAUX

NUM_VIN	MILLESIME	DEGRE
1	1970	12
2	1980	14

→ Ajout du tuple (4, 1950,14) via la vue, conséquence ?

Mise à Jour

```
CREATE VIEW DEGRE_MOYEN_PAR_CRU (CRU, DEGRE_MOYEN)
AS SELECT   CRU, AVG(DEGRE)
FROM        VINS
GROUP BY    CRU;
```

Table VINS

NUM_VIN	CRU	MILLESIME	DEGRE
1	Bordeaux	1970	12
2	Bordeaux	1980	14
3	Beaujolais	2010	11

Vue DEGRE_MOYEN_PAR_CRU

CRU	DEGRE_MOYEN
Bordeaux	13
Beaujolais	11

→ Ajout du tuple (Muscadet, 11) via la vue, conséquence ?

Autre EXEMPLE

Multi-relations

```
CREATE VIEW BUVEURS_BEAUJOLAIS  
    (NUM_BUVEUR, NOM, QUANTITE_COMMANDEE)  
AS SELECT    NUM_BUVEUR, NOM, SUM (C.QUANTITE)  
    FROM      BUVEURS, VINS, COMMANDES  
    WHERE     NUM_BUVEUR = BUVEUR AND  
              VIN = NUM_VIN AND  
              CRU = 'Beaujolais'  
    GROUP BY NUM_BUVEUR, NOM;
```

→ Quels sont les problèmes de mise à jour sur cette vue ?

Principes des mises à jour sur une vue

- La m.a.j via une vue génère éventuellement beaucoup de valeurs nulles
- On peut améliorer les m.a.j via un système de règles avec trigger (do instead) chez PostgreSQL
Dans la "vraie vie" : des triggers "standards"
- Les règles de m.a.j dépendent du SGBD
- Vérification : clause **WITH CHECK OPTION** à la création (à suivre slide 22)
- Matérialisation des vues dans les entrepôts de données → gérer les m.a.j

Traitement par le SGBD - Réécriture

- Modification de la requête appliquée au niveau du code source de la requête
- La requête posée sur une vue est transformée en une requête posée sur les relations de base

CONCEPTEUR

```
CREATE VIEW VINS_05(NUM_VIN, CRU, DEGRE)  
AS SELECT NUM_VIN, CRU, DEGRE  
FROM VINS  
WHERE MILLESIME = 2005 ;
```

SGBD

```
SELECT NUM_VIN, CRU, DEGRE  
FROM VINS  
WHERE MILLESIME = 2005 AND CRU <> 'Fronsac' ;
```

UTILISATEUR

```
SELECT *  
FROM VINS_05  
WHERE CRU <> 'Fronsac' ;
```

Traitement par le SGBD - Restructuration algébrique

- Fusion des arbres algébriques de la vue et de la requête utilisateur
 - Optimisation de l'arbre
- (Nécessite une compilation préalable des vues)

DROITS & INTÉGRITÉ

Droits sur les VUES en SQL

- **Création d'une vue**

- Permis à tout usager ayant au moins le droit SELECT sur toutes les relations intervenant dans la définition de la vue.

- **Propriétaire d'une vue**

- Toujours le droit SELECT
- Droit de Mise à Jour si :
 - La vue peut-être mise à jour
 - Droit de MAJ sur toutes les relations intervenant dans la vue

- **Droits + vues**

- Bonne protection des données

Vues et CIs

La clause **CHECK OPTION** sur un CREATE VIEW (SQL86)

EXEMPLE : CI Domaine

```
CREATE VIEW VINS_OK  
AS SELECT *  
FROM VINS  
WHERE DEG > 8 AND DEG < 14  
WITH CHECK OPTION;
```

EXEMPLE : CI Référentielle

```
CREATE VIEW PRODUCTIONS_OK  
AS SELECT P.*  
FROM PRODUCTIONS P  
WHERE P.NV IN ( SELECT NV  
FROM VINS)  
AND P.NV IN (SELECT NVT  
FROM VITIS)  
WITH CHECK OPTION;
```

EXEMPLE

Table VINS

NUM_VIN	CRU	MILLESIME	DEGRE
1	Bordeaux	1970	12
2	Bordeaux	1980	14
3	Beaujolais	2010	11

Vue VINS_BORDEAUX_BIS

NUM_VIN	CRU	MILLESIME
1	Bordeaux	1970
2	Bordeaux	1980

→ Ajout du tuple (4, Muscadet, 2000) via la vue, conséquence si :

- Sans la clause With Check Option ?
- Avec la clause With Check Option ?

Autre cas

Autre cas : Intégration de schéma à partir de vues

Permettre plusieurs visions venant de SGBD différents au sein d'un même modèle (suite à un rachat d'entreprise, filialisation,...)

Les conflits connus

- Problèmes de terminologie : homonymie / synonymie (problème de niveaux d'abstraction ou de point de vue : client / fournisseur pour achat / vente)
- Incompatibilités de contraintes en fonction des règles de gestion (comité d'entreprise : majeur / mineur, ...)
- Conflit de structure : même entité (personne) mais caractéristiques différentes suivant l'application
- Conflit de représentation : attribut vs. entité dans les modèles conceptuels de haut niveau.

Conclusions

Synthèse sur les vues

- Les tuples d'une vue n'existent pas physiquement
- Les tuples sont calculables par la requête définissant la vue
- La matérialisation peut être effectuée
- Le cliché n'est pas une vue mais une photo statique de la base à un moment donné

Synthèse sur les vues (suite)

Avantages

- Adaptation aux applications utilisateur
- Intégration d'application existantes
- Dynamique du schéma de la base
- Confidentialité des données
- Définition de contraintes d'intégrité

Inconvénients

- Mises à jour difficiles ou impossibles
- D'éventuels problèmes d'efficacité

Nous avons vu les éléments de base concernant les niveaux logiques conceptuel et externe, étape suivante :

➔ NIVEAU INTERNE (INTRODUCTION ET GESTION DE LA CONCURRENCE)